Universitatea Națională de Știință și Tehnologie
POLITEHNICA București

Facultatea de Electronică Telecomunicații și Tehnologia Informației

P0 roadmap

Students:

Nancă Cristian
Petriia Iulian-Nicolae
Stan Alexandru-Gabriel

***Group**: 441Gb*

Project chosen – 6 – LTE vs 5G NR

0.1. Roadmap 3
0.2. Allocation of the tasks around the project 4
I. Feasible 4
II. Specific 4
a. Hardware allocation 4
a. Simulator version & Build Resolution 4
b. 5G-LENA Module Integration 5
III. P0 evidence 5
Output of cttc-nr-demo (./ns3 run cttc-nr-demo) 6
Output of lena-simple-epc (./ns3 run lena-simple-epc) 6
IV. Risks with fallbacks + de pus problema care ne-a generat al doilea cod 7
V. Preliminary Code snippets explanations for midterm phase 7
1. Defining the channel parameters 7
VI. P1 evidence 9
1. Simulation File Architecture 10
VII. P2 evidence 10
1. A brief summary and simulation overview 10
2. Results for the comparison between LTE vs. 5G NR for Mixed Traffic (20 Sensors, 5 Videos) 11

GitHub repository:
https://github.com/petriianicolae/DataCommunication_441G_6

0.1. Roadmap

Environment setup and baseline verification

Week 1 and week 2

Configure the virtual box with `Ubuntu 26.04 (6vCPUs, 9.2GB RAM)`; build stable '`ns-3.48`' via CMake and run some initial tests. Clone the CTTC 5G-LENA (namely branch '`5g-lena-v5.1.py`') and check some dependencies (Eigen3, SQLite); executed 'lena-simple-epc' (for LTE) and 'cttc-nr-demo' (for 5G NR)

-- what was delivered? -- successful build with zero failling tests.

P1 and P2 + midterm bar

Weeks 3-5

Develop the main 'C^{++}' code to incorporate the scenarious (node count, speeds, RAT type, random seeds); integrate 'FlowMonitor' to export network metrices directly to '.csv' files; run the evaluation scenarious (20 sensors, 5 video UEs, 1 seed); identify bottlenecks (NR As-is), and diagnose high latency and severe packet drops in default '5G' settings;

-- what was delivered? -- P1 and P2 results

Documentation in advance for midterm step

Week 6

Creating the documentantion and update of the GitHub repository accordingly to the files created so far.

-- what was delivered? -- documentation (including a small presentation and the updated repository)

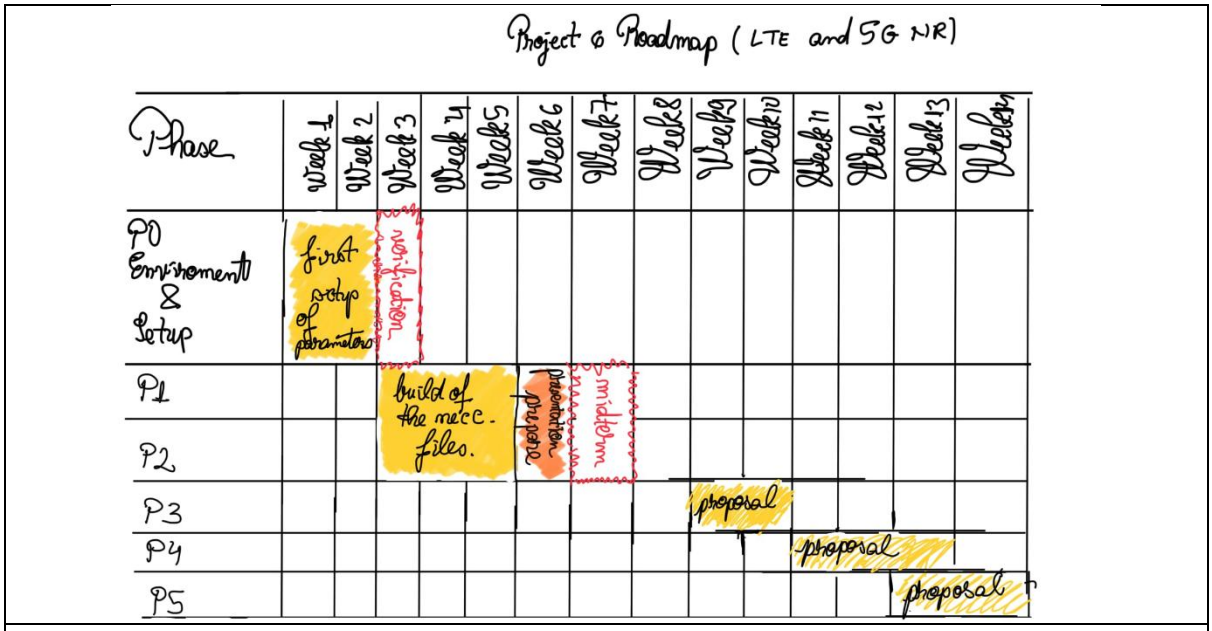


Fig. 0 – Project 6 Roadmap – 'what is done so far and what is proposed to be done in the future'

0.2. Allocation of the tasks around the project[1]

Nancă Cristian *Responsible for Phase P1 and P2*	Petriia Iulian-Nicolae *Responsible for Phase P0*	Stan Alexandru-Gabriel *Responsible for project documentation*
• Application layer traffic generation (2Hz UDP sensors and 2Mbps video flows); • Export the useful pieces of information into '.csv' files; • Static baseline performance evaluation (LTE Base vs. NR As-is vs. NR fixed); • Diagnostic analysis identifying NR scheduling bottlenecks.	• VirtualBox environment first setup (Ubuntu 26.04.1 LTS, 6 vCPUs, 9.2 GB RAM); • ns-3.48 build resolution and stable enviroment configuration; • 5G-LENA (v5.1.y) module integration and dependency validation (Eigen3, SQLite); • Validation runs for baseline examples ('cttc-nr-demo' and 'lena-simple-epc'); • Post-processing for the files created during 'P2' phase.	• P0 and Midterm report structure, layout; • Post-proccesing for the files created during 'P1' phase; • Risk analysis, hardware resource allocation and roadmap updates.

I. Feasible

We will follow the scenario defaults, as we have not yet encountered any significant hurdles that would force us to pivot. This includes confirmed handover, successful runs of lena-simple-epc and cttc-nr-demo. The given roadmap remains unchanged.

II. Specific

a. Hardware allocation

All experiments are conducted on an isolated virtual machine configured using Oracle VM VirtualBox (v7.1.10 r169112) running Ubuntu 26.04.1 LTS. The VM is allocated 6 virtual cores from a 13th Gen Intel Core™ i7-1355U processor, 9.2 GiB of dedicated RAM, and 26.8 GB of disk space. This allocation conforms directly to the project guideline constraint (8–16 GB RAM laptop) while leaving sufficient host overhead.

a. Simulator version & Build Resolution

We started by reading the documentation for the ns-3, namely 'https://www.nsnam.org/docs/installation/html/quick-start.html' and we have run into a problem related to the version for the ns-3 from their GitHub (accessed on **06.10.2026** – '$ git clone https://gitlab.com/nsnam/ns-3-dev.git'). There was a problem within the main function that did not allow the build in function to be executed directly. An alternative identified was to use directly the version https://www.nsnam.org/releases/ns-3-48/documentation.

[1] Each member contribution can be seen also in the GitHub for each part individually using the history option.

To avoid build instability identified on the bleeding-edge upstream development branch (ns-3-dev), we aligned our setup with the recommended stable distribution release ns-3.48. NS-3 was configured via CMake (`./ns3 configure --enable-examples --enable-tests`) with runtime asserts and logging enabled. System integrity was verified by running ./test.py, which yielded `812 of 812 tests passed (0 failed, 0 skipped).`

b. 5G-LENA Module Integration

For 5G New Radio support, the CTTC 5G-LENA module was cloned into `contrib/nr` on branch `5g-lena-v5.1.y`, satisfying the Project 6 prerequisite of version ≥ 5.0 required for handover modelling. All critical numerical and linear algebra dependencies–specifically Eigen3 (for MIMO matrix processing) and SQLite–were verified as active during the build configuration.

III. P0 evidence

5G NR Baseline (cttc-nr-demo): Built and executed without errors. Captured UDP flows showed expected performance: Flow 1 (10.24 Mbps offered load) achieved 10.24 Mbps throughput with 0.276 ms mean delay; Flow 2 (102.40 Mbps offered load) achieved 102.23 Mbps throughput with 0.901 ms mean delay (aggregate mean delay: 0.588 ms).

LTE EPC Baseline (lena-simple-epc): Built and executed successfully, validating standard LTE PHY/MAC/RLC logging via the generation of discrete uplink and downlink trace logs (DlMacStats.txt, UlPdcpStats.txt, DlRsrpSinrStats.txt, etc.).

Output of cttc-nr-demo (./ns3 run cttc-nr-demo)

```
Flow 1 (1.0.0.2:49153 -> 7.0.0.2:1234) proto UDP
Tx Packets: 6000
Tx Bytes:   768000
TxOffered:  10.240000 Mbps
Rx Bytes:   767744
Throughput: 10.236587 Mbps
Mean delay:  0.276044 ms
Mean jitter:  0.030032 ms
Rx Packets: 5998
Flow 2 (1.0.0.2:49154 -> 7.0.0.3:1235) proto UDP
Tx Packets: 6000
Tx Bytes:   7680000
TxOffered:  102.400000 Mbps
Rx Bytes:   7667200
Throughput: 102.229333 Mbps
Mean delay:  0.900967 ms
Mean jitter:  0.119907 ms
Rx Packets: 5990



Mean flow throughput: 56.232960
Mean flow delay: 0.588505
```

Output of lena-simple-epc (./ns3 run lena-simple-epc)

From this scenario test we have identified a couple of files that were generated for this scenario that are listed in the GitHub folder. The primary analysis of these logs shows that for the topology and connectivity, 2 active cells (cellId 1 and 2), each serving one UE (IMSI 1 and 2, RNTI 1), for the channel and physical layer, a pristine radio channel condition (SINR $SINR \cong 1.64 \cdot 10^6$ linear, approx. 62.15dB with with negligible interference (1.35×10^{-20} W), allowing the scheduler to maintain the highest MCS index (mcs = 28) across both Downlink and Uplink without hybrid ARQ retransmissions (rv = 0, ndi = 1, correct = 1). For the transport and data plane (PDCP/RLC), downlink (DL) Uniform delivery of 1,054-byte PDUs at $\approx$ 3.53 ms delay (RLC delay $=$ 3.00 ms) with 0% packet loss (12 Tx $=$ 12 Rx PDUs per UE over the simulation interval) and for the Uplink (UL), Stable transmission of 1,054-byte PDUs at $\approx$ 11.93–12.26 ms delay (RLC delay $=$ 4.71 ms, aggregating two PDCP SDUs into 2,112-byte RLC PDUs).

Layer / Metric	LTE EPC Baseline (lena-simple-epc)	5G NR Baseline (cttc-nr-demo) TXT
Topology	2 eNodeBs, 2 UEs (1 UE / cell)	1 gNodeB, 2 UEs (2 UDP Flows)

Channel Quality	SINR ≈ 62.15 dB, Interf. ≈ 1.35 × 10^{-20} W	Urban Macro default, 3.5 GHz
Modulation / Coding	Max MCS 28 (DL TB: 2,196 B, UL TB: 2,292 B)	Adaptive NR MCS tables
Packet Delivery Ratio (PDR)	100.0% (DL: 24/24 PDUs, UL: 24/24 PDUs)	99.97% (Flow 1), 99.83% (Flow 2)
End-to-End Delay	3.53 ms (DL PDCP), 12.04 ms (UL PDCP)	0.276 ms (Flow 1), 0.901 ms (Flow 2)
Transit / Jitter	RLC Delay: 3.00 ms (DL), 4.71 ms (UL)	Mean Jitter: 0.030 ms (F1), 0.120 ms (F2)

IV. Risks with fallbacks + de pus problema care ne-a generat al doilea cod

Because we previously had 6GB the memory of the virtual toolbox we had to increase the reserved RAM memory up to 9.2 GB. NR simulations operating at 100 MHz channel bandwidth with 100 UEs are exceptionally memory intensive. Running parallel batch execution scripts on standard development laptops (8–16 GB RAM) triggered OS Out-Of-Memory (OOM) process terminations.

Furthermore, because new ns-3 versions appear quite often with new features and properties we have decided to stick only to the ns-3 3.48 version. The project mandates using 5G-LENA v5.0 or later for the NR module in ns-3 to support handovers. Handover features in 5G-LENA are version-sensitive and require an exact corresponding ns-3 core release.

Moreover, the mandatory matrix consists of 55 total runs. High-density UE NR runs can exceed 30 minutes per execution. Our first run exceeded 1 hour. We encountered also a problem related to NR, namely the high 3.5 GHz propagation losses, improper slot formatting.

V. Preliminary Code snippets explanations for midterm phase

Our implementation attempts to follow the 3GPP TR 38.901 industry standard as closely as possible.

1. Defining the channel parameters

Initially, we implemented the simulation with 2 separate channels: one specific for LTE, and one specific for 5G. However, the results we obtained suggested that LTE had a better performance than 5G on all accounts, which intuitively seemed wrong. After analysing the results and debugging the code, we came to a consensus that the reason why LTE seemed better than 5G was because of the differences between the default ns-3 implementation of an LTE channel, and the nr 5G-Lena implementation. As such we have decided to define our own channel structure class. Here are a few of the most important code snippets of this class (called P6UMaLoss), which have been shortened for readability's sake:

Here is the function in which we define the links between the BS (base station), and the UE (user equipment). The "link" here is the connection between the 2, which has to account from losses.

```
static void DrawLinks(uint32_t nUe, uint32_t nSite)
{
    Ptr<UniformRandomVariable> u = CreateObject<UniformRandomVariable>();
    u->SetStream(kStreamLinkLos);                  // stream: same for LTE and NR
    Ptr<NormalRandomVariable> z = CreateObject<NormalRandomVariable>();
    z->SetStream(kStreamLinkShadow);
    ...
    for (uint32_t i = 0; i < nUe; ++i)
        for (uint32_t b = 0; b < nSite; ++b)
        {
            s_losU[i][b] = u->GetValue();
            s_shadowZ[i][b] = z->GetValue();
        }
}
```

These losses come from 2 factors: the presence of LoS (Line of Sight) and shadowing, which are defined below.

```
static bool IsLos(uint32_t ue, uint32_t site, double d2D)
{
    return s_losU.at(ue).at(site) < LosProbability(d2D);   // P_LOS = 18/d + e^(-d/63)(1
- 18/d)
}

static double ShadowingDb(uint32_t ue, uint32_t site, bool los)
{
    return (los ? 4.0 : 6.0) * s_shadowZ.at(ue).at(site);  // sigma = 4 dB LOS, 6 dB NLOS
}
```

For the rest of the losses, we took the formula specified in Table 7.4.1-1 of the TR 38.901 standard, more specifically the formula for „Urban Macro" (UMa).

UMa	LOS	$PL_{UMa-LOS} = \begin{cases} PL_1 & 10m \le d_{2D} \le d'_{BP} \\ PL_2 & d'_{BP} \le d_{2D} \le 5\text{km}, \text{ see note 1} \end{cases}$ $PL_1 = 28.0 + 22\log_{10}(d_{3D}) + 20\log_{10}(f_c)$ $PL_2 = 28.0 + 40\log_{10}(d_{3D}) + 20\log_{10}(f_c) - 9\log_{10}((d'_{BP})^2 + (h_{BS} - h_{UT})^2)$	$\sigma_{SF} = 4$	$1.5m \le h_{UT} \le 22.5m$ $h_{BS} = 25\text{m}$ See note 7
	NLOS	$PL_{UMa-NLOS} = max(PL_{UMa-LOS}, PL'_{UMa-NLOS})$, for $10m \le d_{2D} \le 5\text{km}$ $PL'_{UMa-NLOS} = 13.54 + 39.08\log_{10}(d_{3D}) + 20\log_{10}(f_c) - 0.6(h_{UT} - 1.5)$	$\sigma_{SF} = 6$	$1.5m \le h_{UT} \le 22.5m$ $h_{BS} = 25\text{m}$ Explanations: see note 3
		Optional $PL_{UMa-NLOS} = 32.4 + 20\log_{10}(f_c) + 30\log_{10}(d_{3D})$	$\sigma_{SF} = 7.8$	

Fig. V-1 – The table entry for UMa path loss, taken from TR 38.901[2]

A snippet of our implemented code based on that formula:

[2] *ETSI TR 138 901 V19.1.0 (2025-10) 36 3GPP TR 38.901 version 19.1.0 Release 19, Page 36/164*
https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/19.01.00_60/tr_138901v190100p.pdf

```
const double dBp = 4.0 * (hBs - hE) * (hUt - hE) * fcHz / c;      // ~342 m at 2.14 GHz,
~560 m at 3.5 GHz
if (d2D <= dBp)
    plLos = 28.0 + 22.0 * std::log10(d3D) + 20.0 * std::log10(fcGhz);
else
    plLos = 28.0 + 40.0 * std::log10(d3D) + 20.0 * std::log10(fcGhz)
            - 9.0 * std::log10(dBp * dBp + (hBs - hUt) * (hBs - hUt));
...
const double plNlos = 13.54 + 39.08 * std::log10(d3D) + 20.0 * std::log10(fcGhz) - 0.6 *
(hUt - 1.5);
return std::max(plLos, plNlos);
```

For the mobility scenario, we simulated movement by defining a vector that stores the values of the position of the "car", as it "moves" from the effective area of one cell (basically a "cell tower") to the next. We even accounted for this imaginary car changing lanes:

```
uePos[i] = Vector(-25.0 - 4.0 * i, (i % 2 == 0) ? 195.0 : 205.0, ueHeight);   //
positions
...
nr->SetHandoverAlgorithmType("ns3::NrA3RsrpHandoverAlgorithm");  // Same for LTE
nr->SetHandoverAlgorithmAttribute("Hysteresis", DoubleValue(hysteresisDb));
nr->SetHandoverAlgorithmAttribute("TimeToTrigger", TimeValue(MilliSeconds(tttMs)));
nr->AddX2Interface(bsNodes);
```

There is a lot more code that could be presented, but to save time here is a high-level schematic of the program:

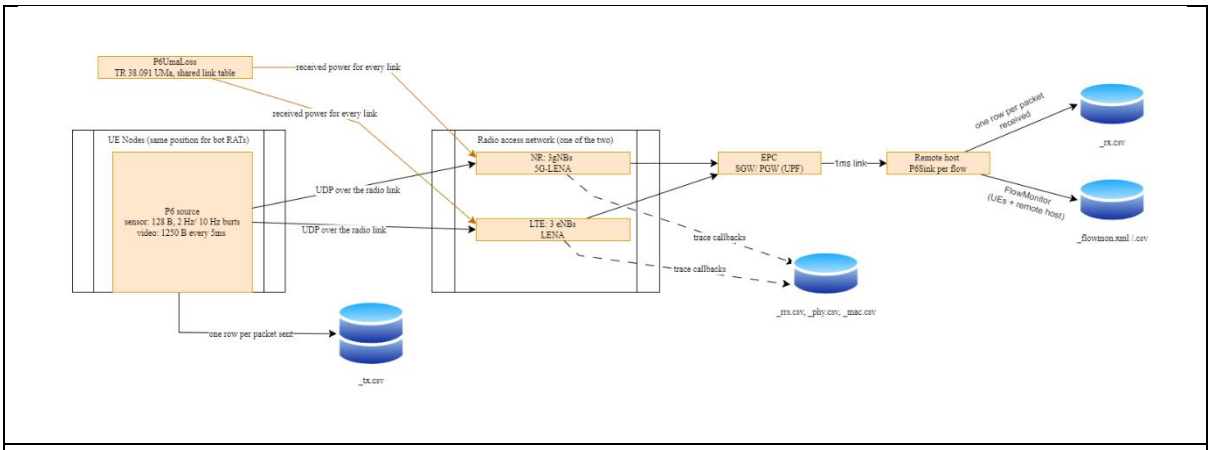


Fig. V – Schematic displaying a top view for the architecture used during the testing scenarios described

VI. P1 evidence

For P1 we analysed the behaviour of LTE networks (20 MHz, 2 GHz) versus 5G NR (100 MHz, 3.5 GHz, μ=1) under a mix of IoT and video traffic. By closely examining metadata, transmission, reception, mobility, and physical layer files, the advantages of 5G are highlighted in terms of lower sensor latency and handover efficiency, as well as the challenges related to signal attenuation at high frequencies in urban environments.

1. Simulation File Architecture

Analysed file / Type	Observations or results that were obtained after P1 phase
'_meta.csv' / Parameters and resources	℘ Defines the physical rules and computational cost; ℘ Confirms that the 5G NR network uses a wider band (100 MHz vs 20 MHz), a higher frequency (3.5 GHz vs 2 GHz), and shorter time slots (μ = 1). Also notes that the 5G simulation is much more complex, taking about 50% more processing time and about 76% more RAM.
'_tx.csv' and '_rx.csv' / Application	℘ They confirm the traffic rates, meaning small occasional bursts for sensors and a steady 2 Mbps stream for video;
'_ues.csv' / Geography and attenuation	℘ Physical map of the users (10 mobile, 15 stationary); ℘ Signal attenuation is much harsher at 3.5 GHz (NR) than at 2 GHz (LTE) for users who are far away or have no direct line of sight.
'_rrc.csv' / Mobility Control	℘ 12 handovers, 0 failures ℘ An advantage of 5G NR, which allows for much faster initial connection and shorter handover interruption time ($\approx 3ms$) in NR vs in LTE($\approx 4.2ms$).
'_phy.csv' / Physical Layer	℘ Can be seen as an explanation for the problem that occurred. In LTE, radio packets are transmitted successfully almost instantly. In NR, the massive attenuation at 3.5 GHz forces video users to endure radio error rates over 30%, generating thousands of retransmissions (HARQ).
'_flowmon.xml / .csv' / Final Performance	℘ Charts for latency, jitter, throughput, and PDR (Packet Delivery Ratio).

VII. P2 evidence

1. A brief summary and simulation overview

This phase involved the static baseline, meaning the comparison between LTE and NR with a group of 20 sensors and 5 videos, including 1 seed. From the folder, '`results\p2`' we have divided the work into 6 core test campaigns:

Test campaign	How was the workload made of?	Objective/ Description
Baseline	20 sensors, 5 videos and 1 seed	Compares LTE base, NR As-is (default), and NR fixed (optimized)
Sizing	Up to 100 sensors, 10 videos	Stress-tests network performance under higher node scaling.
iso	1 sensor, 0 videos	Measures baseline propagation latency without contention.
Combined & diag.	Architectural parameter variations	Evaluates fixes (30 kHz SCS, 100 MHz BW, BSR) bridging As-it to Fixed.
demo	Direct head-to-head setup	Offers a streamlined comparative overview of LTE base vs. NR fixed.

2. Results for the comparison between LTE vs. 5G NR for Mixed Traffic (20 Sensors, 5 Videos)

Metric	Traffic Type	LTE (20 MHz @ 2.0 GHz)	5G NR (100 MHz @ 3.5 GHz, μ=1)	Which technology was better?
Slot Duration	All	1.0 ms	0.5 ms ($\mu = 1$)	5G NR
Mean Latency	IoT Sensors	~13.5 ms	~10.9 ms	5G NR (-2.6 ms)
Mean Latency	Video Users	~8.5 ms	>15.0 ms	LTE (-6.5 ms)
Packet Delivery Ratio (PDR)	Video Users	100.0% (0 drops)	98.7% (326 packets dropped)	LTE
Key Bottleneck / Advantage	Both	Higher latency floor due to 15 kHz SCS	3.5 GHz attenuation causes ~30% HARQ BLER	Trade-off dependent

3. Baseline KPI analysis for 20 sensors and 5 video nodes[3]

Metric	LTE Base	NR As-is (Unoptimized)	NR Fixed (Optimized)	Best Configuration
Sensor Median Latency (p_{50})	13.47 ms	25.18 ms	10.49 ms	NR Fixed (-22% vs. LTE)
Sensor Latency Tail (p_{95})	Baseline floor	30.58 ms	Optimized	NR Fixed
Sensor Packet Delivery Ratio (PDR)	100.00%	94.94% (~5% loss)	98.08%	LTE Base
Worst-Case Edge UE PDR	High stability	77.60% (starvation)	Recovered	LTE Base / NR Fixed
Video Loss Ratio	0.00%	5.91%	3.45%	LTE Base

[3] The results obtained from 'kpi_runs.csv'.

4. Scalability and network sizing[4]

Metric	LTE Base	NR As-is (Unoptimized)	NR Fixed (Optimized)	Key Observation / Winner
Sensor Median Latency (p_{50})	14.60 ms	24.90 ms	10.41 ms	NR Fixed maintains flat low-latency profile under load.
Sensor Tail Latency (p_{95})	15.90 ms	30.50 ms	16.13 ms	LTE Base & NR Fixed demonstrate tight delay bounds.
Sensor Packet Delivery Ratio (PDR)	100.00%	90.50%	95.81%	LTE Base handles high contention without losing sensor packets.
Video Loss Ratio	0.22%	>21.00%	11.97%	LTE Base handles raw bandwidth contention better at scale.
Worst-Case Node Reliability	High stability	~70.00% PDR (Collapse)	Stable	NR As-is completely collapses under maximum density.

5. Diagnostic and the path to 'NR fixed'

Parameter / Layer	Unoptimized (NR As-is) Bottleneck	Applied Fix / Optimization (NR Fixed)	Technical Impact & Result
Numerology & Bandwidth	Standard SCS ($\mu = 0$, 15 kHz) over narrow carrier.	Upgraded to $\mu = 1$ (30 kHz SCS) & widened channel to 100 MHz.	Halved slot duration to 0.5 ms, enabling sub-11 ms median sensor latencies.
TDD Slot Formatting	Unbalanced frame pattern causing directional collision.	Implemented DDDFU TDD pattern.	Perfectly balances asymmetric traffic (heavy video DL vs. continuous sensor UL).
BSR & Scheduling Logic	Unoptimized Buffer Status Reports starving UEs.	Tuned BSR evaluation logic at the gNB MAC layer.	Eliminates resource starvation; cures the 77.6% worst-case UE PDR floor.
Physical Layer (PHY)	High Uplink first-transmission BLER.	Reduced retransmissions via MAC/PHY alignment.	Drastically cuts packet corruption and HARQ delay overhead.

[4] Up to 100 Sensors, 10 Video Nodes

ANNEX 1 – Results for P2 phase

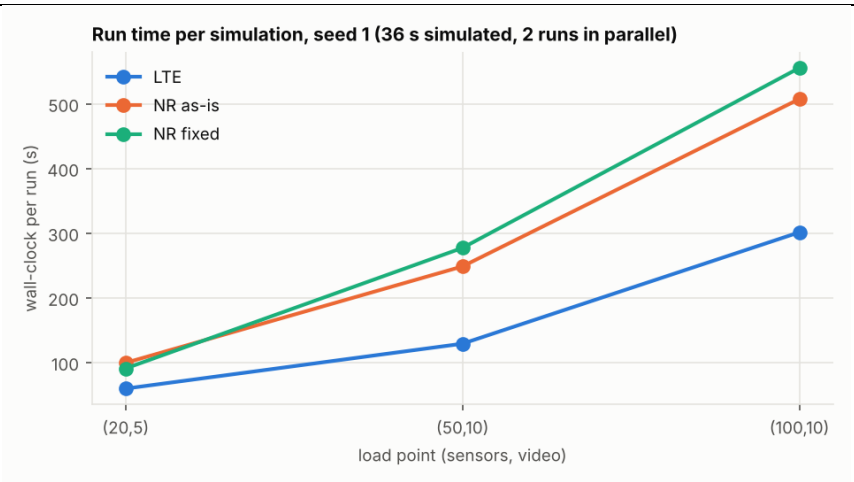


Fig.VII.1 – The runtime per simulation with one seed

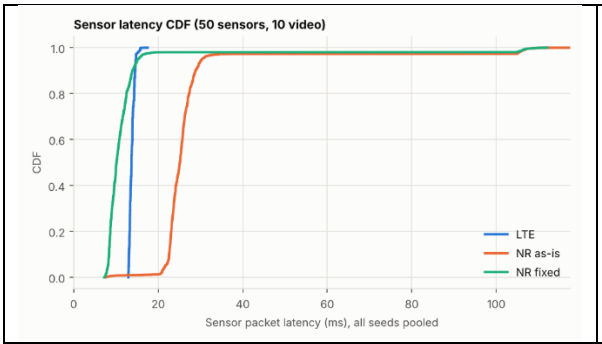


Fig.VII.2 – Sensor latency CDF for 50 sensors, 10 video

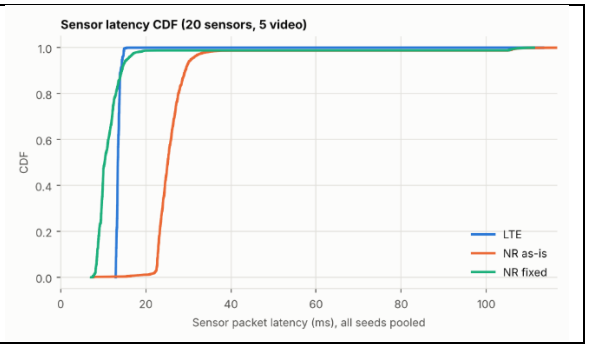


Fig.VII.3 – Sensor latency CDF for 20 sensors, 5 video

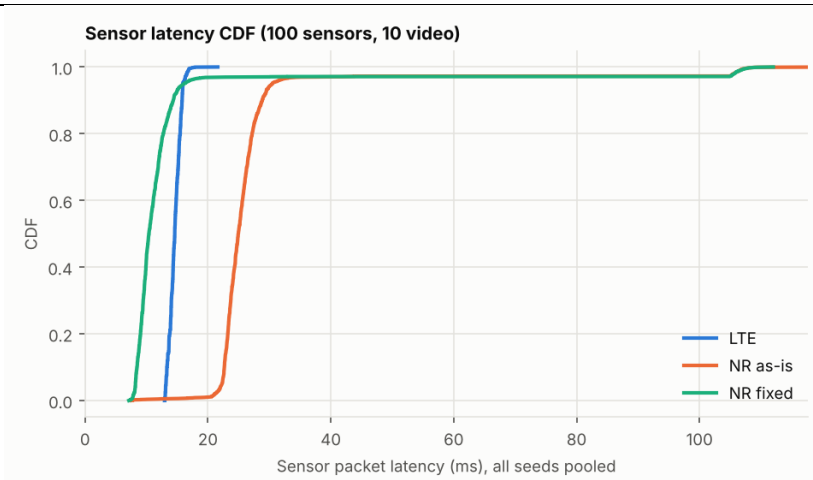


Fig.VII.4 – Sensor latency CDF for 100 sensors, 10 video

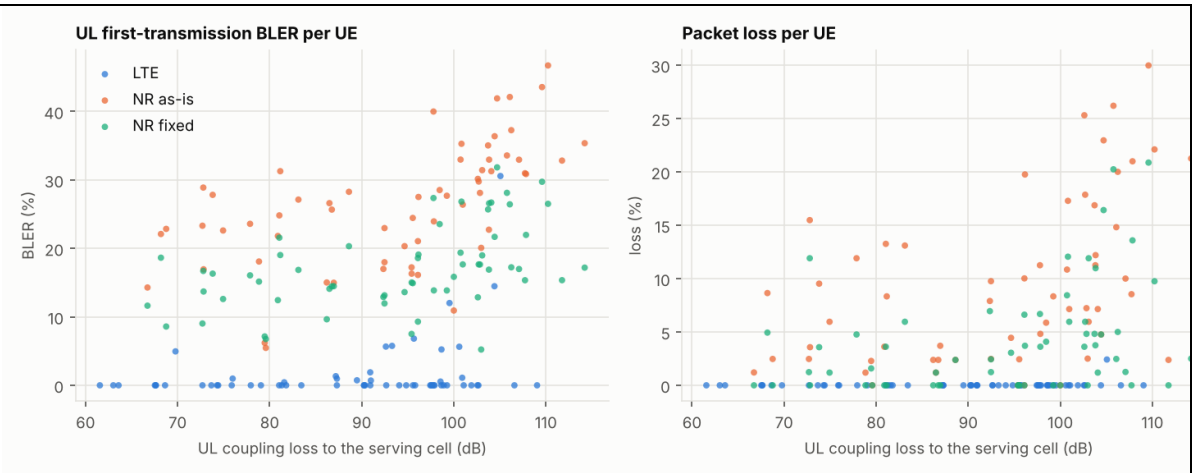


Fig.VII.5 – Packet loss per UE for the 3 cases that we encountered

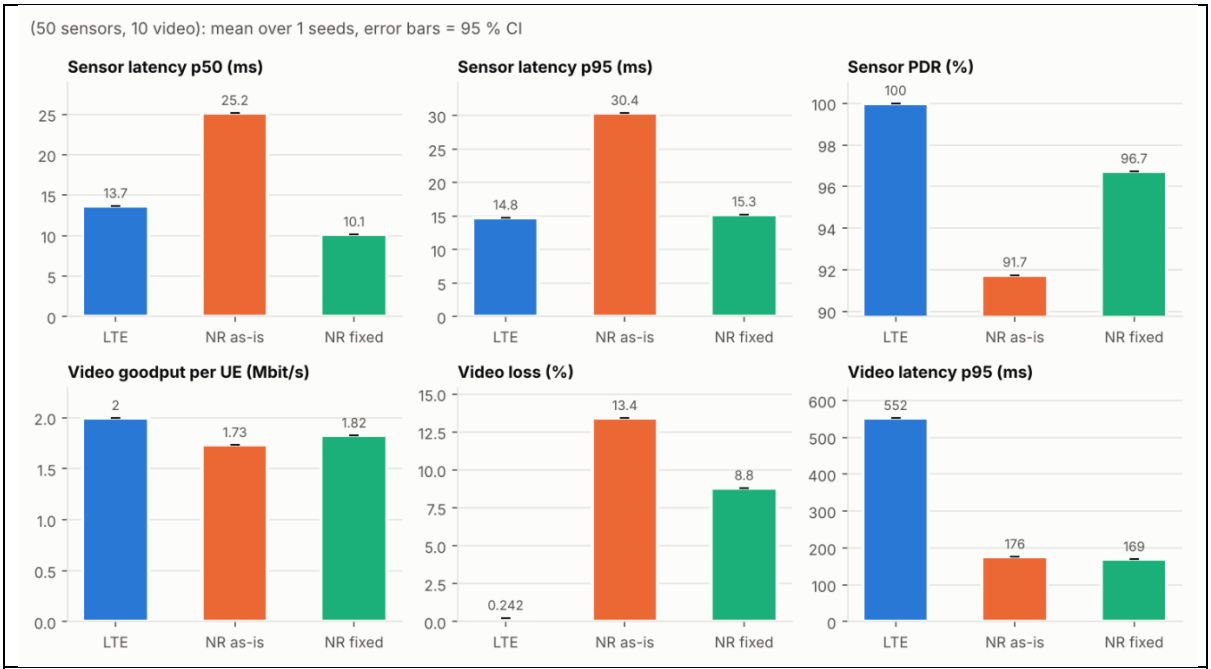


Fig.VII.6 – Key Performance Indicator for the case of 50 sensors and 10 videos

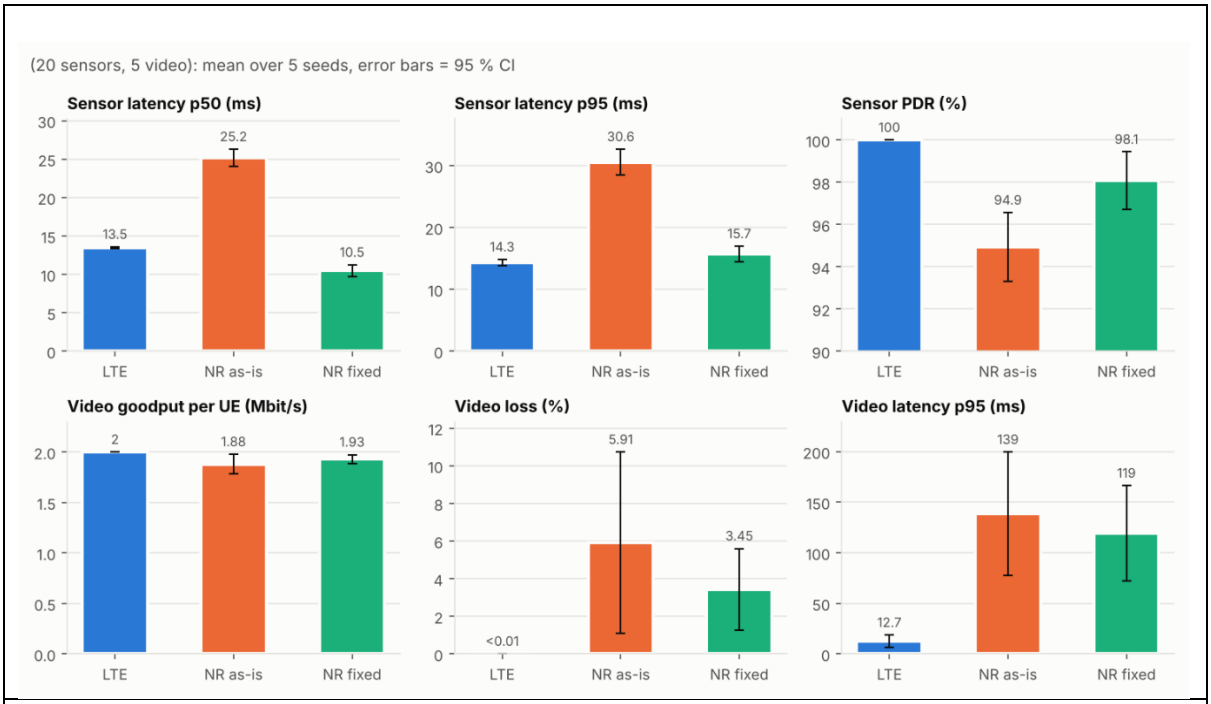


Fig.VII.7 – Key Performance Indicator for the case of 20 sensors and 5 videos

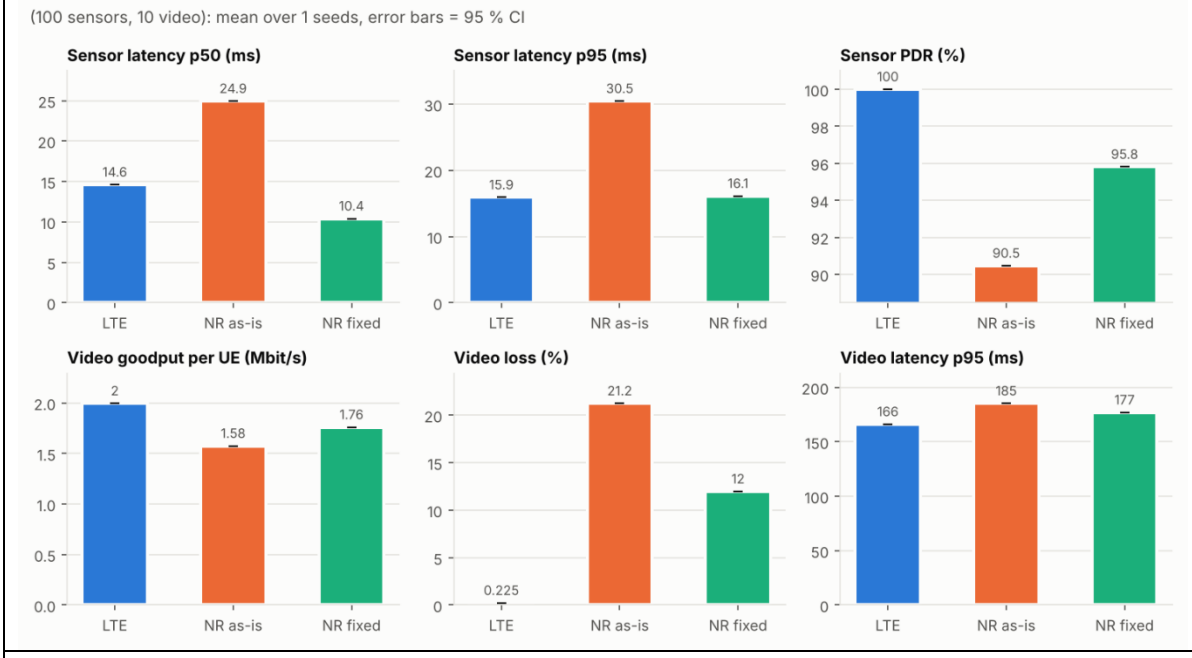


Fig.VII.8 – Key Performance Indicator for the case of 100 sensors and 10 videos

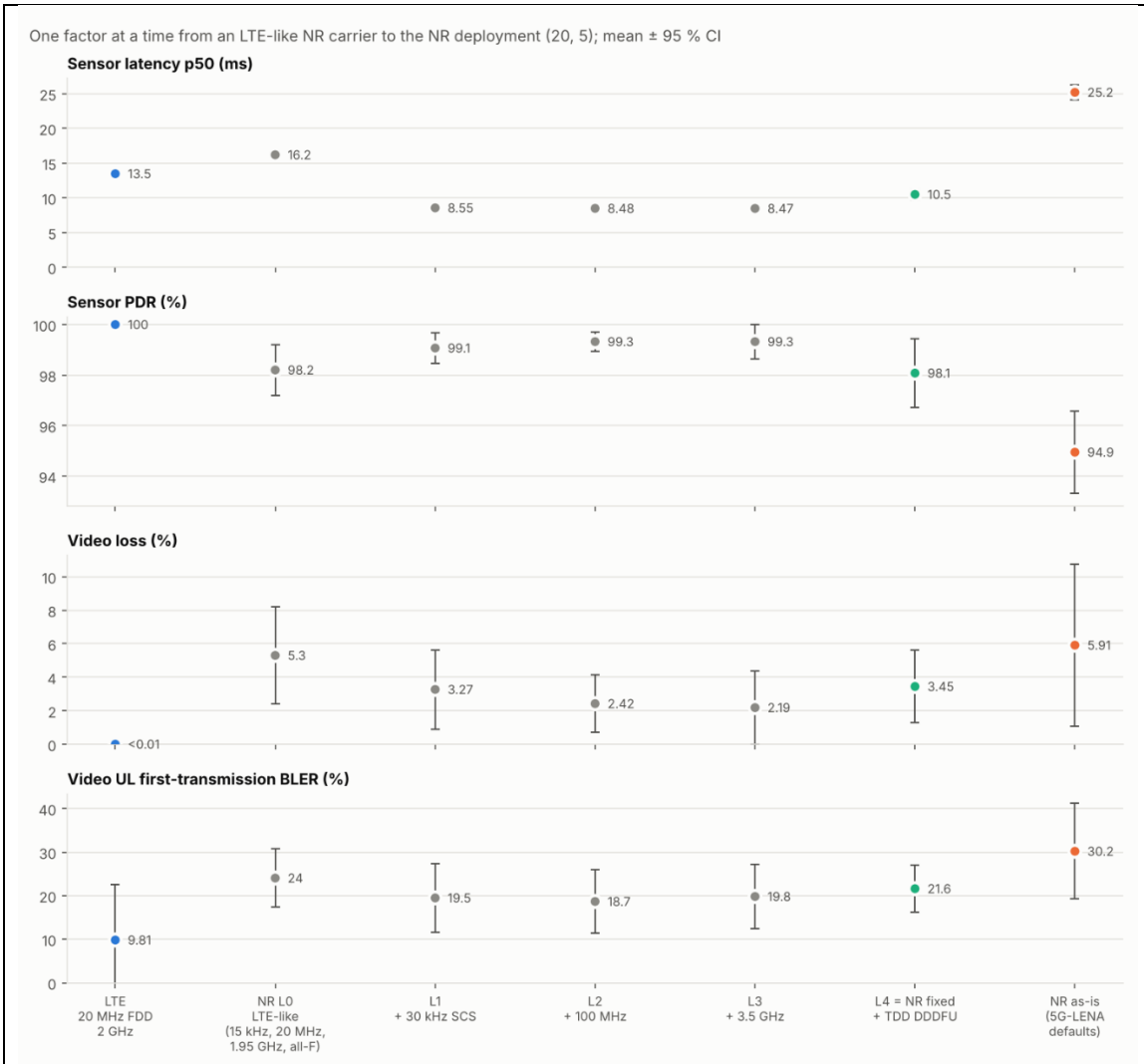


Fig.VII.9 – Relevant figures for the sensor latency p50, sensor PDR, video loss, video UL first-transmission BLER